

Rockchip Developer Guide Linux GMAC DPDK

ID: RK-KF-YF-487

Release Version: V1.0.0

Release Date: 2023-03-06

Security Level: ☐Top-Secret ☐Secret ☐Internal ☒Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD.("ROCKCHIP")DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document provides an introduction to the use of DPDK for the Ethernet GMAC interface on the Rockchip platform.

Product Version

Chipset	Kernel Version
RK3568	4.19+

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	** Date**	Change Description
V1.0.0	Wu Dachao	2023-03-06	Initial Version

Contents

Rockchip Developer Guide Linux GMAC DPDK

1. Code Compilation
 - 1.1 Kernel
 - 1.2 Compiling DPDK
2. Running DPDK Programs
 - 2.1 Mounting HugePages
 - 2.2 Loading KO
 - 2.3 Setting Performance Mode
 - 2.4 Running testpmd
 - 2.5 Running l2fwd
 - 2.6 Running l3fwd
3. Pktgen
 - 3.1 Download pktgen-dpdk Source Code
 - 3.2 Compiling DPDK
 - 3.3 Compiling Pktgen
 - 3.4 Running Pktgen Program

1. Code Compilation

1.1 Kernel

- First, enable the UIO node in DTS with reference to RK3568-evb1:

```
diff --git a/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dtsi
b/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dtsi
index 0cb57e9d8529..c7729258e51d 100644
--- a/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dtsi
@@ -262,6 +262,14 @@
         status = "okay";
     };

+&gmac_uio0 {
+    status = "okay";
+};
+
+&gmac_uio1 {
+    status = "okay";
+};
+
/*
 * power-supply should switch to vcc3v3_lcd1_n
 * when mipi panel is connected to dsi1.
```

- Compile the Kernel Module (KO)

```
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 rockchip_linux_defconfig
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 menuconfig, configure CONFIG_UIO=m,
CONFIG_STMMAC_UIO=m, CONFIG_HUGETLBFS=y
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 rk3568-evb1-ddr4-v10-linux.img -j8
Flash boot.img
adb push drivers/uio/uio.ko, adb push drivers/net/ethernet/stmicro/stmmac/stmmac_uio.ko
```

1.2 Compiling DPDK

The system running on the development board used for DPDK testing is Debian 11, with DPDK version 21.11. For compilation, refer to http://doc.dpdk.org/guides-21.11/linux_gsg/cross_build_dpdk_for_arm64.html.

- Cross-compilation on PC

```
wget https://developer.arm.com/-/media/Files/downloads/gnu-a/9.2-2019.12/binrel/gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu.tar.xz
tar -xvf gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu.tar.xz
export PATH=$PATH:<cross_install_dir>/gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu/bin

apt install build-essential
apt install pkg-config-aarch64-linux-gnu
apt-get install python3 python3-pip
apt install meson
apt install ninja-build

meson aarch64-build-gcc --cross-file config/arm/arm64_armv8_linux_gcc
meson --reconfigure aarch64-build-gcc --cross-file config/arm/arm64_armv8_linux_gcc -
Dbuildtype=debug -Dplatform=arm64 -Dexamples=l2fwd,l3fwd
ninja -C aarch64-build-gcc
```

- Compiling DPDK on the development board

```
apt-get install python3 python3-pip
apt install meson
apt install ninja-build
apt-get install libdpkg-perl
apt-get install build-essential
apt-get install python3-pyelftools

meson -Ddisable_drivers='common/cnxk' build
cd build
ninja
ninja install
```

2. Running DPDK Programs

2.1 Mounting HugePages

```
echo 1024 > /sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages
```

2.2 Loading KO

```
insmod uio.ko  
insmod stmmac_uio.ko
```

By default, the network traffic after booting up still uses the native kernel network. Here, we achieve switching between DPDK and the native kernel network by insmodding and rmmodding the `stmmac_uio.ko` module. That is, after unloading the KO module, it will revert to the original kernel network state.

- Entering DPDK Network Control:

```
root@linaro-alip:/home/linaro# insmod stmmac_uio.ko  
[ 41.232761] Generic PHY stmmac-1:00: attached PHY driver [Generic PHY]  
(mii_bus:phy_addr=stmmac-1:00, irq=POLL)  
[ 41.236447] dwmac4: Master AXI performs any burst length  
[ 41.236497] rk_gmac-dwmac fe010000.ethernet eth0: No Safety Features support found  
[ 41.236886] rockchip_eth_uio_drv fe010000.uio: Registered uio_eth0 uio devices, 3  
register maps attached  
[ 41.241453] Generic PHY stmmac-0:00: attached PHY driver [Generic PHY]  
(mii_bus:phy_addr=stmmac-0:00, irq=POLL)  
[ 41.257084] dwmac4: Master AXI performs any burst length  
[ 41.257138] rk_gmac-dwmac fe2a0000.ethernet eth1: No Safety Features support found  
[ 41.257523] rockchip_eth_uio_drv fe2a0000.uio: Registered uio_eth1 uio devices, 3  
register maps attached
```

- Returning to Kernel Network Control:

```
root@linaro-alip:/home/linaro# rmmod stmmac_uio  
[ 2200.304636] Generic PHY stmmac-0:00: attached PHY driver [Generic PHY]  
(mii_bus:phy_addr=stmmac-0:00, irq=POLL)  
[ 2200.318163] dwmac4: Master AXI performs any burst length  
[ 2200.318205] rk_gmac-dwmac fe2a0000.ethernet eth1: No Safety Features support found  
[ 2200.318227] rk_gmac-dwmac fe2a0000.ethernet eth1: IEEE 1588-2008 Advanced Timestamp  
supported  
[ 2200.318443] rk_gmac-dwmac fe2a0000.ethernet eth1: registered PTP clock  
[ 2200.319414] IPv6: ADDRCONF(NETDEV_UP): eth1: link is not ready  
[ 2200.322971] Generic PHY stmmac-1:00: attached PHY driver [Generic PHY]  
(mii_bus:phy_addr=stmmac-1:00, irq=POLL)  
[ 2200.324883] dwmac4: Master AXI performs any burst length  
[ 2200.324921] rk_gmac-dwmac fe010000.ethernet eth0: No Safety Features support found
```

```
[ 2200.324945] rk_gmac-dwmac fe010000.ethernet eth0: IEEE 1588-2008 Advanced Timestamp supported
[ 2200.325468] rk_gmac-dwmac fe010000.ethernet eth0: registered PTP clock
[ 2200.325814] IPv6: ADDRCONF(NETDEV_UP): eth0: link is not ready
[ 2205.385406] IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready
```

2.3 Setting Performance Mode

```
echo performance > /sys/devices/system/cpu/cpufreq/policy0/scaling_governor
echo performance > /sys/devices/system/cpu/cpufreq/policy4/scaling_governor
echo performance > /sys/devices/system/cpu/cpufreq/policy6/scaling_governor
echo performance > /sys/class/devfreq/dmc/governor
```

2.4 Running testpmd

- Forwarding Mode Test Commands
 - `--vdev=net_stmmac0 --vdev=net_stmmac1` indicates the specified virtual devices, which currently have fixed names.
 - `--main-lcore=0` signifies that core 0 is used for management, while cores 2 and 3 are used for forwarding.
 - `--iova-mode=pa` because the device does not support iommu, hence iova-mode is set to pa mode.
 - `--` is used to separate eal parameters from testpmd parameters.
 - `-i` indicates entering the interactive mode of the dpdk-testpmd command.

```
root@linaro-alip:/home/linaro# ./dpdk-testpmd --iova-mode=pa --vdev=net_stmmac0 --
vdev=net_stmmac1 -l 0,2,3 --main-lcore=0 -- -i
EAL: Detected CPU lcores: 4
EAL: Detected NUMA nodes: 1
EAL: Detected static linkage of DPDK
EAL: Multi-process socket /var/run/dpdk/rte/mp_socket
EAL: Selected IOVA mode 'PA'
TELEMETRY: No legacy callbacks, legacy socket not created
Interactive-mode selected
Warning: NUMA should be configured manually by using --port-numa-config and --ring-numa-
config parameters along with --numa.
testpmd: create a new mbuf pool <mb_pool_0>: n=163456, size=2176, socket=0
testpmd: preferred mempool ops selected: ring_mp_mc
Configuring Port 0 (socket 0)
stmmac_net: stmmac_eth_link_update()Port (0) link is Up
Port 0: BA:A0:3F:FD:B2:8C
Configuring Port 1 (socket 0)
stmmac_net: stmmac_eth_link_update()Port (1) link is Up
Port 1: B6:A0:3F:FD:B2:8C
Checking link statuses...
stmmac_net: stmmac_eth_link_update()Port (0) link is Up
stmmac_net: stmmac_eth_link_update()Port (1) link is Up
```

Done

- Starting Forwarding Mode Test

```
testpmd> start
io packet forwarding - ports=2 - cores=1 - streams=2 - NUMA support enabled, MP allocation
mode: native
Logical Core 2 (socket 0) forwards packets on 2 streams:
RX P=0/Q=0 (socket 0) -> TX P=1/Q=0 (socket 0) peer=02:00:00:00:00:01
RX P=1/Q=0 (socket 0) -> TX P=0/Q=0 (socket 0) peer=02:00:00:00:00:00

io packet forwarding packets/burst=32
nb forwarding cores=1 - nb forwarding ports=2
port 0: RX queue number: 1 Tx queue number: 1
  Rx offloads=0x0 Tx offloads=0x0
  RX queue: 0
    RX desc=0 - RX free threshold=0
    RX threshold registers: pthresh=0 hthresh=0 wthresh=0
    RX Offloads=0x0
  TX queue: 0
    TX desc=0 - TX free threshold=0
    TX threshold registers: pthresh=0 hthresh=0 wthresh=0
    TX offloads=0x0 - TX RS bit threshold=0
port 1: RX queue number: 1 Tx queue number: 1
  Rx offloads=0x0 Tx offloads=0x0
  RX queue: 0
    RX desc=0 - RX free threshold=0
    RX threshold registers: pthresh=0 hthresh=0 wthresh=0
    RX Offloads=0x0
  TX queue: 0
    TX desc=0 - TX free threshold=0
    TX threshold registers: pthresh=0 hthresh=0 wthresh=0
    TX offloads=0x0 - TX RS bit threshold=0
```

- Viewing Forwarding Data:

```
testpmd> show port stats all

##### NIC statistics for port 0 #####
RX-packets: 5240160   RX-missed: 0       RX-bytes: 314409600
RX-errors: 0
RX-nombuf: 0
TX-packets: 0         TX-errors: 0       TX-bytes: 0

Throughput (since last show)
Rx-pps:      0         Rx-bps:      0
Tx-pps:      0         Tx-bps:      0
#####

##### NIC statistics for port 1 #####
RX-packets: 0         RX-missed: 0       RX-bytes: 0
RX-errors: 0
```

```

RX-nombuf: 0
TX-packets: 5180224    TX-errors: 0    TX-bytes: 310813440

Throughput (since last show)
Rx-pps: 0    Rx-bps: 0
Tx-pps: 0    Tx-bps: 0
#####
testpmd> show port stats all

##### NIC statistics for port 0 #####
RX-packets: 6382336    RX-missed: 0    RX-bytes: 382940160
RX-errors: 0
RX-nombuf: 0
TX-packets: 0    TX-errors: 0    TX-bytes: 0

Throughput (since last show)
Rx-pps: 681809    Rx-bps: 327268480
Tx-pps: 0    Tx-bps: 0
#####

##### NIC statistics for port 1 #####
RX-packets: 0    RX-missed: 0    RX-bytes: 0
RX-errors: 0
RX-nombuf: 0
TX-packets: 6322397    TX-errors: 0    TX-bytes: 379343820

Throughput (since last show)
Rx-pps: 0    Rx-bps: 0
Tx-pps: 681810    Tx-bps: 327269160
#####

```

- Other Common Settings:

- `--nb-cores` indicates the number of cores that dpdk-testpmd uses for forwarding work.
- `--rxq` is for receiving queue descriptors.
- `--txq` is for transmitting queue descriptors.
- `--rxq` specifies the number of receive queues for dpdk-testpmd, with RK3568 having a queue count of 1.
- `--txq` specifies the number of transmit queues for dpdk-testpmd, with RK3568 having a queue count of 1.

2.5 Running l2fwd

l2fwd requires at least two cores to test packet forwarding by default.

```

./dpdk-l2fwd -l 0,2,3 --main-lcore=0 --iova-mode=pa --vdev=net_stmmac0 --vdev=net_stmmac1 -
- -q 1 -p 0x3

```


The console information of l3fwd refreshes every 10 seconds, which may cause packet loss. It is recommended to redirect the console information to a file.

2.6 Running l3fwd

```
./dpdk-l3fwd -l 3 -n 1 --iova-mode=pa --vdev=net_stmmac0 --vdev=net_stmmac1 -- -p 0x3 -P --  
config="(0,0,3),(1,0,3)" --parse-ptype
```

- -p PortMask parameter specifies the port mask used for the network interfaces
- -P parameter indicates that all network interfaces are set to promiscuous mode to receive all packets
- --config (port,queue,lcore),[(port,queue,lcore)] parameter is used to configure the correspondence between network interfaces, queues, and cores, for example, --config (0,0,3) means that queue 0 of network interface 0 is processed by core 3

It is worth noting that the above output prints the default routing rules of l3fwd, that is,

```
LPM: Adding route 198.18.0.0 / 24 (0) [net_stmmac0]  
LPM: Adding route 198.18.1.0 / 24 (1) [net_stmmac1]  
LPM: Adding route 2001:200:: / 64 (0) [net_stmmac0]  
LPM: Adding route 2001:200:0:1:: / 64 (1) [net_stmmac1]
```

This means that packets destined for the IP range 198.18.0.0/24 will be forwarded through network interface 0, and packets destined for the IP range 198.18.1.0 / 24 will be forwarded through network interface 1. The above default routing rules are configured in the source code, so when testing l3fwd, it is necessary to set the destination IP and source IP of the test data correctly.

3. Pktgen

Running Pktgen on the board is based on DPDK, so it is necessary to first compile and install DPDK. For compilation, refer to Section 1.2 on compiling DPDK for the development board.

3.1 Download pktgen-dpdk Source Code

```
git clone http://dpdk.org/git/apps/pktgen-dpdk  
apt-get install libpcap-dev  
apt-get install libnuma-dev  
apt install meson  
apt install ninja-build pkg-config
```

3.2 Compiling DPDK

```
cd build
ninja
ninja install
ldconfig
export PKG_CONFIG_PATH=/usr/local/lib/aarch64-linux-gnu/pkgconfig/
```

3.3 Compiling Pktgen

```
cd pktgen-dpdk
meson build
cd build
ninja
```

3.4 Running Pktgen Program

```
./build/app/pktgen --iova-mode=pa --vdev=net_stmmac0 -l 6,7 --proc-type auto --log-level
debug -- -P -m 7.0
```

In the EAL options section, you can refer to DPDK EAL parameters. The most important parameter is the `-l` parameter, which is used to specify the list of cores to be used, for example: `-l 1,2` or `-l 1-2`, indicating the use of cores 1 and 2.

It is worth noting that at least two cores must be specified for pktgen, as pktgen requires one core to interact with the user, such as responding to user inputs during the testing process.

The most important parameter in the pktgen's own parameter section is the `-m` parameter, which is used to specify the correspondence between network ports and cores, for example:

`-m 2.0`: This means that core 2 is responsible for processing network port 0. It is important to note that if you want to specify multiple correspondences (using multiple network cards and multiple cores), you need to use the `-m` parameter multiple times.

If you want to receive packets, it is best to specify the `-P` parameter as well, indicating that all network ports should enter promiscuous mode to receive all data packets.

Set packet format and start Pktgen:

```
set 0 size 64
set 0 src ip 198.18.0.100/24
set 0 dst ip 198.18.1.101
set 0 dst mac ba:a0:3f:fd:b2:8c
start 0
```